POLYTECH NANTES | Pôle Sciences et technologie Nantes Université

Rapport de Languages matériels et réutilisation

ETN5 - SETR

Élèves :

Nolan BUCHET

Maxence BOULEAU

Date :

Octobre 2026

Table des matières

1 Introduction **2**

2 Récupération d'un modèle de RAM **3**
2.1 Validation du fonctionnement de la RAM 3
2.2 Modification du code de la RAM 3
2.3 Ajout de la généricité de la RAM 3
2.4 Ajout de la fonctionnalité du Chip Select 4

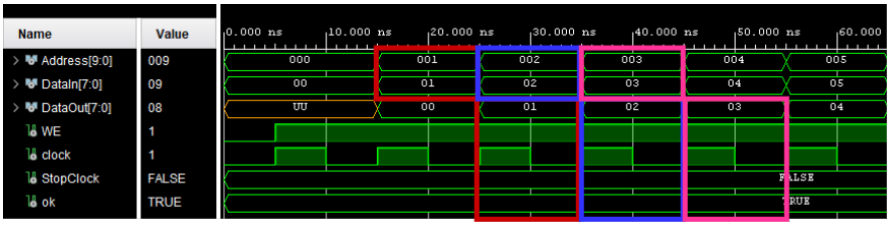
1 Introduction

Ce mini-projet vise à concevoir et valider par simulation un système numérique sur puce articulé autour d'un processeur RISC-V et d'une mémoire modulaire, tout en mettant en application les principes de généricité et de réutilisation de blocs matériels (IP cores). La démarche consiste d'abord à transformer une mémoire RAM élémentaire en VHDL pour la rendre paramétrable et à instancier un bloc mémoire étendu (quadram) associé à un décodeur d'adresses. Dans un second temps, le modèle de processeur décrit en Verilog est adapté pour supporter l'extension d'instructions RV32IM et pour s'interfacer avec la mémoire externe. L'ensemble est enfin réuni au sein d'une entité englobante mixte VHDL/Verilog puis validé sous ModelSim à l'aide de programmes de test en assembleur, illustrant ainsi l'intégralité du flot de co-conception et d'intégration système.

2 Récupération d'un modèle de RAM

2.1 Validation du fonctionnement de la RAM

Nous récupérons le modèle `ramchip.vhd` et son testbench afin de pouvoir observer et analyser son comportement sur ModelSim.



Dans le chronogramme ci-dessus, nous observons que notre simulation représente correctement le comportement d'une RAM.

Le bon fonctionnement se valide en commençant par mettre `WE` à 1. Sans celui-ci, la RAM ne peut pas écrire de données. Ensuite, à chaque front d'horloge, nous mettons à jours une addresse dans laquelle nous voulons écrire une donnée. Nous mettons ensuite à jour la donnée que nous voulons écrire dans la RAM.

- A 20 ns : Ecriture de 01 à l'addresse 001 -> DataOut = 01
- A 30 ns : Ecriture de 02 à l'addresse 002 -> DataOut = 02
- A 40 ns : Ecriture de 03 à l'addresse 003 -> DataOut = 03, et ainsi de suite

Un effet de décalage apparait, la valeur affichée correspond toujours à la donnée appliquée au cycle précédent. Exemple entre 20 ns et 30 ns, Address vaut 001, DataIn vaut 01, et DataOut restitue 00. Celui ci est normal car notre Ram est synchrone, et ne restitue donc la donnée qu'au cycle suivant.

2.2 Modification du code de la RAM

Dans un second temps, nous mettons à jour le code en remplaçant les `std_logic_vector` par des `unsigned` afin de pouvoir effectuer des opérations arithmétiques sur les adresses. Le fonctionnement reste identique à celui de la RAM initiale.

2.3 Ajout de la généricité de la RAM

Pour rendre notre RAM plus générique, nous ajoutons des paramètres de taille de données et d'adresses. Nous ajoutons donc les généricités `DATA_LENGTH` et `ADDR_LENGTH` qui permettront de définir la taille de la RAM.

```
1 entity sync_ram is
2   generic (
```

```
    ADDR_LENGHT : integer := 10;
    DATA_LENGHT : integer := 8
  );
  port (
    clock   : in  std_logic;
    we      : in  std_logic;
    address : in  unsigned((ADDR_LENGHT - 1) downto 0);
    datain  : in  unsigned((DATA_LENGHT - 1) downto 0);
    dataout : out unsigned((DATA_LENGHT - 1) downto 0)
  );
end entity sync_ram;
```

Listing 1 – Nouvelle entité de la RAM

2.4 Ajout de la fonctionnalité du Chip Select